character sequences are used [5, 6]. In this chapter, however, the focus is on language models over natural language words or wordlike units, which we define for now as units delimited by whitespace. We first present the fundamental n-gram modeling approach to statistical language modeling, as well as a range of more advanced modeling techniques, before discussing problems arising from the characteristics of particular languages, such as morphologically rich languages or languages without explicit word segmentation. The chapter concludes with a presentation of multilingual and crosslingual language modeling approaches.

5.2. n-Gram Models

The probability of a word sequence W of nontrivial length cannot be computed directly because unrestricted natural language permits an infinite number of word sequences of variable lengths. The probability $P(W)$ can be decomposed into a product of component probabilities according to the chain rule of probability:

$$P(W) = P(w_1 \dots w_t) = P(w_1) \prod_{i=1}^{t} P(w_i | w_{i-1} w_{i-2} \dots w_2 w_1) \tag{5.1}$$

Because the individual terms in this product are still too difficult to be computed directly, statistical language models make use of the ***n*-gram approximation**, which is why they are also called n-gram models. The assumption is that all previous words except for the $n - 1$ words directly preceding the current word are irrelevant for predicting the current word, or, alternatively, that they are equivalent. Given this assumption of "history equivalence classes," the n-gram model is defined as:

$$P(W) \approx \prod_{i=1}^{t} P(w_i | w_{i-1}, \dots w_{i-n+1}) \tag{5.2}$$

Depending on the length of n, we can distinguish between unigrams ($n = 1$), bigrams ($n = 2$), trigrams ($n = 3$), or 4-grams, 5-grams, and so on. An n-gram model is also often called an $(n - 1)$-th order Markov model, because

the approximation in Equation 5.2 embodies the Markov assumption of the independence of the current word given all other words except the $n - 1$ preceding words.

5.3. Language Model Evaluation

Before describing parameter estimation methods and further refinements of the basic n-gram modeling approach, let us consider the problem of judging the performance of a language model. According to the basic definition given previously, a language model computes the probability of a word sequence W. How can we tell whether a language model is successful at estimating word sequence probabilities? Typically, two criteria are used: **coverage rate** and **perplexity** on a held-out test set that does not form part of the training data. The coverage rate measures the percentage of n-grams in the test set that are represented in the language model. A special case of this is the out-of-vocabulary rate (or OOV rate), which is 100 minus the unigram coverage rate, or, in other words, the percentage of unique word types *not* covered by the language model. The second criterion, perplexity, is an informationtheoretic measure. Given a model p of a discrete probability distribution, perplexity can be defined as 2 raised to the entropy of p:

$$PPL(p) = 2^{H(p)} = 2^{-\sum_x p(x) log_2 p(x)} \tag{5.3}$$

In language modeling, we are often more interested in the performance of a language model q on a test set of a fixed size, say t words ($w_1 w_2 \ldots w_t$). Then, language model perplexity can be computed as

$$PPL(p,q) = 2^{H(p,q)} = 2^{-\sum_{i=1}^{t} p(w_i) log_2 q(w_i)} \tag{5.4}$$

or simply

$$2^{-\frac{1}{t}\sum_{i=1}^{t} log_2 q(w_i)} \tag{5.5}$$

where $q(w_i)$ computes the probability of the *i'th* word. If $q(w_i)$ is an n-gram probability, the equation becomes

$$2^{-\frac{1}{t}\sum_{i=1}^{t} log_2 p(w_i | w_{i-1}, \ldots, w_{i-n+1})} \tag{5.6}$$

When comparing different language models, especially models based on different ways of decomposing a text into language modeling units (e.g., words vs. morphemes), care must be taken to normalize their perplexities with respect to the same number of units in order to obtain a meaningful comparison.

Perplexity can be thought of as the average number of equally likely successor words when transitioning from one position in the word string to the next. If the model has no predictive power at all, perplexity is equal to the vocabulary size. A model achieving perfect prediction, by contrast, has a perplexity of one. The goal in language model development is most often to minimize the perplexity on a held-out data set representative of the domain of interest.

However, it should be noted that sometimes the goal of language modeling is not to predict word sequence probabilities but to distinguish between "good" and "bad" word sequence hypotheses generated by some frontend such as a machine translation system or a speech recognizer. In this case, the language model should assign maximally distinct scores to word sequences that are errorful, ungrammatical, or otherwise unacceptable, as opposed to those that are correct. Optimizing for minimum perplexity does not necessarily achieve this goal; we return to this point in Section 5.6.3.

5.4. Parameter Estimation

5.4.1. Maximum-Likelihood Estimation and Smoothing

The standard procedure in training n-gram models is to estimate n-gram probabilities using the maximum-likelihood criterion in combination with parameter smoothing. The maximumlikelihood estimate is obtained by simply computing relative frequencies:

$$P(w_i|w_{i-1}, w_{i-2}) = \frac{c(w_i, w_{i-1}, w_{i-2})}{c(w_{i-1}, w_{i-2})} \tag{5.7}$$

where $c(w_i, w_{i-1}, w_{i-2})$ is the count of the trigram $w_{i-2}w_{i-1}w_i$ in the training data. It is obvious that this method fails to assign nonzero probabilities to word sequences that have not been observed in the training data; on the other hand, the probability of sequences that were observed might be overestimated. The process of redistributing probability mass such that peaks in the *n*-gram probability distribution are flattened and zero estimates are floored to some small nonzero value is called **smoothing**. The most common smoothing technique is **backoff**. Backoff involves splitting *n*-grams into those whose counts in the training data fall below a predetermined threshold τ and those whose counts exceed the threshold. In the former case, the maximum-likelihood estimate of the *n*-gram probability is replaced with an estimate derived from the probability of the lower-order (*n* − 1)-gram and a backoff weight. In the latter case, *n*-grams retain their maximum-likelihood estimates, discounted by a factor that redistributes probability mass to the lower-order distribution. Thus, the backed-off probability P_{BO} for w_i given w_{i-1}, w_{i-2} is computed as follows:

$$P_{BO}(w_i|w_{i-1}, w_{i-2}) = \begin{cases} d_c P(w_i|w_{i-1}, w_{i-2}) \text{ if } c > \tau \\ \alpha(w_{i-1}, w_{i-2}) P_{BO}(w_i|w_{i-1}) \text{ otherwise} \end{cases} \quad (5.8)$$

where *c* is the count of (w_i, w_{i-1}, w_{i-2}), and d_c is a discounting factor that is applied to the higher-order distribution. The normalization factor $\alpha\ (w_{i-1},\ w_{i-2})$ ensures that the entire distribution sums to one and is computed as

$$\alpha(w_{i-1}, w_{i-2}) = \frac{1 - \sum_{w_i : c(w_i, w_{i-1}, w_{i-2}) > \tau} d_c P(w_i|w_{i-1}, w_{i-2})}{\sum_{w_i : c(w_i, w_{i-1}, w_{i-2}) \leq \tau} P_{BO}(w_i|w_{i-1})} \quad (5.9)$$

The way in which the discounting factor is computed determines the precise smoothing technique. Well-known techniques include Good-Turing, Witten-Bell, Kneser-Ney, and others; see the study by Chen and Goodman [7] for an in-depth description and comparison of various smoothing techniques. For example, in Kneser-Ney smoothing, a fixed discounting parameter *D* is applied to the raw *n*-gram counts before computing the probability estimates:

$$P_{KN}(w_i|w_{i-1},w_{i-2}) = \begin{cases} \frac{max\{c(w_i,w_{i-1},w_{i-2})-D,0\}}{\sum_{w_i} c(w_i,w_{i-1},w_{i-2})} \text{ if } c > \tau \\ \alpha(w_{i-1},w_{i-2}) P_{KN}(w_i|w_{i-1}) \text{ otherwise} \end{cases} \quad (5.10)$$

In modified Kneser-Ney smoothing, which is one of the most widely used techniques, different discounting factors D_1, D_2, D_{3+} are used for n-grams with exactly one, two, or three or more counts:

$$Y = \frac{n_1}{n_1 + 2 * n_2} \quad (5.11)$$

$$D_1 = 1 - 2Y\frac{n_2}{n_1} \quad (5.12)$$

$$D_2 = 2 - 3Y\frac{n_3}{n_2} \quad (5.13)$$

$$D_{3+} = 3 - 4Y\frac{n_4}{n_3} \quad (5.14)$$

where n_1, n_2, ... are the counts of n-grams with one, two, ..., counts.

Another common way of smoothing language model estimates is linear model interpolation [8]. In linear interpolation, M models are combined by

$$P(w_i|w_{i-1},w_{i-2}) = \sum_{m=1}^{M} \lambda_m P(w_i|h_m) \quad (5.15)$$

where λ is a model-specific weight. The component models may use different conditioning variables, such as histories of different lengths, or they may have been estimated from different data sets, such as a large set of general data or a smaller set of domain-specific data (see Section 5.5). The following constraints hold for the model weights: $0 \le \lambda \le 1$, and $\sum_m \lambda_m = 1$. Weights are estimated by maximizing the log-likelihood (minimizing the perplexity) on a held-out data set that is different from the training set for the component models (and also different from the final evaluation or test set). This is typically done using the expectation-maximization (EM) procedure [9].

5.4.2. Bayesian Parameter Estimation

Bayesian probability estimation is an alternative parameter estimation method whereby the set of parameters of a model

is itself viewed as a random variable governed by a prior statistical distribution. Given a training sample S and a set of parameters θ, $P(\theta)$ denotes a prior distribution over different possible values of θ, and $P(\theta|S)$ is the posterior distribution, which can be expressed using Bayes's rule as

$$P(\theta|S) = \frac{P(S|\theta)P(\theta)}{P(S)} \tag{5.16}$$

In language modeling, the set of parameters is the vector of word probabilities, that is, $\theta = \langle P(w_1), \ldots, P(w_K)\rangle$ (where K is the vocabulary size) for a unigram model, or, more generally, $\theta = \langle P(w_1|h_1), \ldots, P(w_K|h_K)\rangle$ for an n-gram model with K n-grams and history h of a specified length. The training sample S is a sequence of words, $w_1 \ldots w_t$. We require a point estimate of θ given the constraints expressed by the prior distribution and the training sample. This can be done using either the maximum a posteriori (MAP) criterion or the Bayesian criterion. The former finds the value that maximizes the posterior probability given in Equation 5.16:

$$\theta^{MAP} = \underset{\theta\in\Theta}{\operatorname{argmax}}\, P(\theta|S) = \underset{\theta\in\Theta}{\operatorname{argmax}}\, P(S|\theta)P(\theta) \tag{5.17}$$

where Θ is the space of all possible assignments for θ. The Bayesian criterion finds the *expected* value of θ given the sample S:

$$\theta^B = E[\theta|S] = \int_\Theta \theta P(\theta|S)d\theta \tag{5.18}$$

$$= \frac{\int_\Theta \theta P(S|\theta)P(\theta)d\theta}{\int_\Theta P(S|\theta)P(\theta)d\theta} \tag{5.19}$$

Under the assumption that the prior distribution is a uniform distribution, the MAP estimate of the probability for a given word w is equivalent to the maximum-likelihood estimate, whereas the Bayesian estimate is equivalent to the maximum-likelihood estimate with Laplace smoothing:

$$\theta^B_w = \frac{c(w)+1}{\sum_w c(w) + K} \tag{5.20}$$

Different choices for the prior distribution lead to different estimation functions. The most commonly used prior distribution in language model is the Dirichlet

distribution. The Dirichlet distribution is the conjugate prior to the multinomial distribution (i.e., prior and posterior distribution have the same functional form). It is defined as

$$p(\theta) = D(\alpha_1, \ldots, \alpha_K) = \frac{\Gamma(\sum_{k=1}^{K} \alpha_k)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_k^{\alpha_k - 1} \tag{5.21}$$

where Γ is the gamma function and $\alpha_1,\ldots,\alpha_K$ are the parameters of the Dirichlet distribution (or hyperparameters), which can also be thought of as counts derived from an a priori training sample. The MAP estimate under the Dirichlet prior is

$$\theta^{MAP} = \underset{\theta \in \Theta}{\operatorname{argmax}} \frac{\Gamma(\sum_{k=1}^{K} \alpha_k)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_k^{n_k + \alpha_k - 1} \tag{5.22}$$

where n_k is the number of times word k occurs in the training sample. The result is another Dirichlet distribution, parameterized by $n_\kappa + \alpha$. The MAP estimate of $P(\theta|W, \alpha)$ thus is equivalent to the maximum-likelihood estimate with add-m smoothing, where $m_k = \alpha_k - 1$; that is, pseudocounts of size $\alpha_\kappa - 1$ are added to each word (or n-gram) count. Hyperparameters offer a convenient way of integrating different information sources into the language model estimation process. This approach has most successfully been used for language model adaptation (e.g., [10]), where the prior is computed from a large out-of-domain data set and the observed counts are computed from a small in-domain data set; see Section 5.5 for further details on Bayesian language model adaptation. Early approaches to building language models entirely based on Bayesian estimation [11] did not perform as well as standard n-gram models estimated with the techniques described in Section 5.4.1. However, with recent progress in Bayesian statistics, alternative models have been developed that yield results comparable to those of Kneser-Ney smoothed n-gram models. In particular, this includes models that assume a latent topic structure of documents and use Bayesian estimation techniques to model this structure. These models are discussed more fully in Section 5.6.8.

5.4.3. Large-Scale Language Models

Recently, there has been much interest in scaling language models to very large data sets. The amount of available monolingual data increases daily, and for many languages, models can now be built from sets as large as several billions or trillions of words. Scaling language models to data sets of this size requires modifications to the ways in which language models are trained, stored, and integrated into real-world systems (e.g., speech recognition decoders). It also affects parameter estimation in that exact probability computations may no longer be feasible.

Several sites [12, 13] have proposed distributed approaches to large-scale language modeling. Their common feature is that the entire language model training data is subdivided into several partitions, and counts or probabilities derived from each partition are stored in separate physical locations (i.e., they are distributed over a cluster of independent compute nodes in a client-server architecture). During runtime, clients can request statistics from a set of data partitions through a language model server, thus producing (possibly interpolated) probability estimates on demand. The advantage of distributed language modeling is that it scales to very large amounts of data and large vocabulary sizes and allows new data to be added dynamically without having to recompute static model parameters. Desired parameters, such as the n-gram model order or the specific mixture of different data partitions, can be chosen and requested at runtime, which allows dynamic decoding approaches to be used. The drawback of distributed approaches, however, is the slow speed of networked queries.

In Brants et al. [13], a nonnormalized form of backoff is introduced that differs from standard backoff (Equation 5.8) in that it uses the raw relative frequency estimate instead of a discounted probability if the n-gram count exceeds the minimum threshold (in this case 0):

$$S(w_i|w_{i-1}, w_{i-2}) = \begin{cases} P(w_i|w_{i-1}, w_{i-2}) \text{ if } c > 0 \\ \alpha S(w_i|w_{i-1}) \text{ otherwise} \end{cases} \quad (5.23)$$

The α parameter is fixed for all contexts rather than being dependent on the lower-order *n*-gram, as in Equation 5.8. The result is no longer a normalized probability distribution but a set of unnormalized scores (denoted by *S* rather than *P* for probabilities) that are used in the same manner as standard probabilities. The advantage of this scheme is that unnormalized scores are easier to compute in a distributed framework because summing over all *n*-gram contexts (which are stored in different physical locations and are therefore expensive to query) is no longer required. Interestingly, the authors found that the model performs almost as well as a model trained with standard Kneser-Ney smoothing on large amounts of data.

An alternative possibility is to use large-scale distributed language models at a secondpass rescoring stage only, after first-pass hypotheses have been generated using a smaller language model [14, 15]. Yet another approach is to store large-scale language models in the working memory of a single machine but to use efficient though possibly lossy data structures. Talbot and Osborne [16] investigate the use of Bloom filters for this purpose. Under this approach, corpus statistics (*n*-gram frequency counts, context counts, etc.) are represented in a highly memory-efficient, randomized data structure (a Bloom filter) in a quantized fashion. If $c(w_1 \ldots w_n)$ is the count of *n*-gram $w_1 \ldots w_n$, the quantized count $q(w_1 \ldots w_n)$ is defined as

$$q(w_1 \ldots w_n) = 1 + [log_b c(w_1 \ldots w_n)] \quad (5.24)$$

At test time, the necessary statistics are queried from the filter, and the true frequency is approximated by the expected count given the quantized count:

$$E[c(w_1 \ldots w_n)|q(w_1 \ldots w_n) = j] = \frac{b^{j-1} + b^j - 1}{2} \quad (5.25)$$

In this framework, frequencies will never be underestimated but may possibly be overestimated, although the probability

of overestimation decreases exponentially with the size of the estimation error. The advantage of this method is that, although the raw frequency counts may be inaccurate, querying the data structure is fast, thus enabling the model to compute smoothed probabilities on the fly. In practice, it was found that the performance of a Bloom filter language model on a machine translation task was close to that of a language model based on exact parameter estimation while providing memory savings of a factor of 4 to 6 [16].

The overall trend in large-scale language modeling is to abandon exact parameter estimation of the type described in the previous section in favor of approximate techniques. This development is likely to continue, leading to more powerful and refined approximation techniques as the number and size of text data collections continue to increase.

5.5. Language Model Adaptation

It is often the case that the amount of language model training data is insufficient, particularly when porting a speech or language processing system to a new domain, topic, or language. For this reason, much effort has been invested in **language model adaptation**, that is, designing and tuning a language model such that it performs well on a new test set for which little equivalent training data is available.

The most commonly used adaptation method is that of mixture language models, or model interpolation. Usually, an in-domain language model is trained using a small amount of in-domain data, and a larger background or generic model is trained using a large amount of out-of-domain data. These models are then interpolated according to Equation 5.15, optimizing the interpolation weights on a small development set. Naturally, this approach generalizes to more than two models, and several variations of basic model interpolation have been developed.

One popular method is topic-dependent language model adaptation. Seymour and Rosenfeld [17] show how documents can first be clustered into a large number of different topics, and individual language models can be built for each topic cluster. The desired final model is then fine-tuned by choosing and interpolating a smaller number of topic-specific language models.

A form of dynamic self-adaptation of a language model is provided by **trigger models**. The idea is that, in accordance with the underlying topic of the text, certain word combinations are more likely than others to co-occur; some words are said to "trigger" others—for example, the words *stock* and *market* in a financial news text. For clustering words according to topic and using them as trigger pairs, latent semantic analysis (LSA) [18] and probabilistic latent semantic analysis (PLSA) [19] have also been been used [20, 21, 22]. LSA, originally formulated for information retrieval, represents a collection of texts as a document-word co-occurrence matrix, where rows correspond to words, columns correspond to documents, and individual cells represent the (possibly weighted) frequency of the word in the document. Singular-value decomposition applied to this matrix maps words to a low-rank continuous vector space. The semantic similarity within this space can then be computed using, for example, the cosine similarity of the corresponding word vectors. A language model can be adapted dynamically as follows:

$$P(w_i|h_i,\widetilde{h}_i) = \frac{P(w_i|h_i)\rho(w_i,\widetilde{h}_i)}{Z(h_i,\widetilde{h}_i)} \tag{5.26}$$

Here, $\widetilde{h}_i$ represents the global document history in LSA space up to word i, and ρ represents the similarity function measuring the compatibility between the current word and the semantic history. The idea is that the probabilities of semantically similar words get boosted by a factor proportional to their similarity with the global document history. Trigger relationships can also be incorporated into a

language model in the form of constraints within constraint-based modeling frameworks (such as the maximum entropy [MaxEnt] model discussed in Section 5.6.5 [23] or the discriminative language model discussed in Section 5.6.3 [24]).

PLSA extends the basic, nonprobabilistic LSA approach by assuming a more sophisticated latent class model to decompose the word-document co-occurrence matrix instead of using simple singular-value decomposition. Given a latent class c, the probability of each word-document co-occurrence (w, d) can be expressed as:

$$P(w,d) = \sum_c P(c)P(w|c)P(d|c) = P(d)\sum_c P(c|d)P(w|c) \tag{5.27}$$

However, a potential concern with PLSA is its tendency to overfit to the training data. A more recent form of topic-based clustering is latent Dirichlet allocation (LDA) [25], which can be interpreted as a regularized version of PLSA. Topic models based on LDA and its extensions are described in Section 5.6.8.

A further variant of the standard adaptation framework is **unsupervised adaptation**, which is primarily relevant for speech recognition applications. Rather than using written text or perfectly transcribed speech as adaptation data, it is possible to directly use the output of a speech recognizer instead [26]. Several studies (e.g., [27, 28]) have shown that this method achieves roughly half of the improvement obtained by using perfect transcriptions for adaptation.

Recently, it has become common to utilize the Internet as a resource for additional language model data. If available text data for a given topic, domain, or language is insufficient, the web can be queried and additional domain-related data can be retrieved. After preprocessing and possibly filtering the data, it is added to the pool of existing data, or a separate model is trained on the web data and is subsequently interpolated with an existing baseline language model.

Several rapid adaptation methods based on this general procedure have been described [29, 30, 31, 32].

Finally, alternative probability estimation schemes for language model adaptation have also been investigated. One of these is **maximum a posteriori (MAP) adaptation** [10].

Here, the counts collected separately from the generic out-of-domain (OD) and the in-domain adaptation (ID) data are combined as follows:

$$P(w|h) = \frac{c_{OD}(w,h) \cdot \varepsilon c_{ID}(w,h)}{c_{OD}(h) \cdot \varepsilon c_{ID}(h)} \tag{5.28}$$

where h and w are the history and predicted word respectively, c_{OD} is the count obtained from the out-of-domain data, and c_{ID} is the count from the in-domain data. The ε parameter ranges between 0 and 1 and represents the weight assigned to the adaptation data: because the amount of out-of-domain data is usually going to outweigh the amount of available adaptation data, the contributions from both sets can be balanced by appropriately setting the ε parameter, which is done empirically. A comparison of MAP and mixture models [33] has shown that mixture models are less robust than MAP adaptation toward variability in the adaptation data.

Although much of the work on language model adaptation has been performed in the context of speech recognition, several studies have also been done on adapting language models for machine translation. In Eck, Vogel, and Waibel [34] and Zhao, Eck, and Vogel [35], queries constructed from first-pass translation hypotheses are used to select additional sentences from a large corpus of target language data, which are then used as additional training data. Models built from this data are interpolated with the baseline language model and are used to retranslate the source language input text. Additional techniques for adapting language models using crosslingual data are described in Section 5.8.2.

5.6. Types of Language Models

Although n-gram models are still the most widely used type of statistical language model by far, a range of alternative models have been developed that have shown additional benefits in practical applications, often when used in combination with n-gram models.

5.6.1. Class-Based Language Models

Class-based language models [36] are a simple way of addressing data sparsity in language modeling. Words are first clustered into classes, either by automatic means [37] or based on linguistic criteria, for example, using part-of-speech (POS) classes. The statistical model makes the assumption that words are conditionally independent of other words given the current word class. If c_i is the class of word w_i, a class-based bigram model can be defined as follows:

$$P(w_i|w_{i-1}) = \sum_{c_i,c_{i-1}} P(w_i|c_i)p(c_i|c_{i-1},w_{i-1})P(c_{i-1}|w_{i-1}) \quad (5.29)$$

$$= \sum_{c_i,c_{i-1}} P(w_i|c_i)P(c_i|c_{i-1})P(c_{i-1}|w_{i-1}) \quad (5.30)$$

under the assumption that c_i is independent of w_{i-1} given c_{i-1}. Usually, a class will contain more than one word, such that the model simplifies to

$$P(w_i|w_{i-1}) = P(w_i|c_i)P(c_i|c_{i-1}) \quad (5.31)$$

Goodman [38] compared this decomposition to the following model:

$$P(w_i|w_{i-1}) \approx P(w_i|c_i,c_{i-1})P(c_i|c_{i-1}) \quad (5.32)$$

where the current word is conditioned not only on the current word class but also on the preceding word classes. Experiments on the North American Business News Corpus (with the training size ranging between 100,000 and 284 million words), using 20,000 test sentences and a vocabulary of 58,000 words showed that the model in Equation 5.32 worked better unless the training data size was at the lower end. Class-based models have been

successful in reducing perplexity as well as practical performance in a wide range of language processing systems; however, they typically need to be interpolated with a word-based language model.

5.6.2. Variable-Length Language Models

In standard language modeling, vocabulary units are defined by simple criteria, such as whitespace delimiters, and the prediction of the probability of the next word is based on an invariable fixed-length history (save for backoff). Several modifications to this basic approach have been developed that aim at redefining vocabulary units in a data-driven way, resulting in merged units composed out of a variable number of basic units. These approaches are termed **variable-length** n-gram models. The challenge in these models is to find the best segmentation of the word sequence $w_1\ w_2\ ...w_t$ into language modeling units in addition to estimating the language model probabilities. Deligne and Bimbot [39] model the segmentation as a hidden variable and use an ME procedure to find the best segmentation. A variable-length model of order 7 yielded slight improvements in perplexity compared to a standard word-based bigram model, but no application results were reported.

A simpler approach is to start with the initial whitespace-based segmentation suggested by the standard orthography of the language and to merge selected units, rather than attempting to rederive the entire vocabulary segmentation from scratch. A limited number of merged units corresponding to frequently observed short phrases can thus be added to the language model vocabulary. A common criterion for identifying potential phrasal candidate units is the mutual information between adjacent words (e.g., [40]). The actual selection of phrasal units is then made using a greedy iterative algorithm: in each iteration, those candidates are selected that reduce the perplexity of a development corpus the most [41, 42]. In Zitouni, Smaili,

and Haton [42], word class information is used to identify candidate phrasal units in that mutual information is computed over pairs of classes rather than pairs of words, which was shown to reduce perplexity by roughly 10% compared to word-based selection of candidate pairs. This model also achieved an 18% relative reduction in worderror rate on a medium-scale French automatic speech recognition (ASR) task.

5.6.3. Discriminative Language Models

Standard n-gram models embody a generative model for assigning a probability to a given word sequence W. However, in practical applications like machine translation or speech recognition, the task of a language model is often to separate good sentence hypotheses from bad sentence hypotheses. For this reason, it would be desirable to train language model parameters discriminatively, such that word strings of widely differing quality receive maximally distinct probability estimates. Recent attempts at such **discriminative language modeling** include Roark et al. [43], Collins, Saraçlar, and Roark [44], Shafran and Hall [45], and Arisoy et al. [46]. Here, the language model is applied to an existing set of competing sentence hypotheses Y generated for an input x (e.g., an acoustic sequence in the case of speech recognition, or the source language string in the case of machine translation) by some generation function GEN(x). Arbitrary feature functions $\varphi(x, y)$ can be defined jointly over the input and each output $y \in Y$. These are then used in a global linear model that selects the best hypothesis:

$$F(x) = \underset{y \in \mathrm{GEN}(x)}{\operatorname{argmax}} \ \phi(x, y)\alpha \tag{5.33}$$

where α is a weight vector. In the most basic case, the feature functions are the raw n-gram counts obtained from the training data. However, additional feature functions representing statistics over word classes or subword units may be integrated (see §5.7.1). The parameter vector α can be trained by the perceptron algorithm [47] or a conditional log-

linear model [43]. The perceptron algorithm, for example, iterates through all training samples (for several epochs) and selects the currently best-scoring hypothesis for each sample. If it differs from the true reference hypothesis, it updates the current weights by adding the counts of the features in the correct hypothesis and subtracting their counts in the chosen hypothesis. Such a training procedure directly minimizes the desired objective function, such as word-error rate in a speech recognition system. Thus, the weights with which the counts of different n-grams contribute to the final model are trained to optimize system performance rather than minimize the perplexity criterion described in Section 5.3. Roark, Saraçlar, and Collins [48] reported a 1.8% absolute improvement in word-error rate (from 39.2% to 37.4%) on a large-vocabulary speech recognition task for a one-pass system and a 0.9% reduction in word-error rate for a multipass recognizer. Recently, discriminative language modeling has also been applied to statistical machine translation [49], where it yielded an improvement of 1 to 2 BLEU points over a state-of-the-art baseline system. As we shall see, a discriminative language model also offers a convenient way of integrating additional linguistic information, such as morphological features.

5.6.4. Syntax-Based Language Models

A well-known drawback of n-gram language models is that they cannot take into account relevant words in the history that fall outside the limited window of the directly preceding $n - 1$ words. However, natural language exhibits many types of **long-distance dependencies**, where the choice of the current word is dependent on words that are relatively far removed in terms of sentence position. In the following example, the plural noun *Investors* triggers the plural verb *were* but is not taken into account as a conditioning variable by an n-gram model, where n is usually no larger than 4 or 5.

Investors, *who still showed confidence in financial markets last week,* **were** *responsible for today's downturn.*

To address this problem, several approaches to syntax-based language modeling have been developed, whose goal is to explicitly model such syntactic relationships and use them to estimate better probabilities. Most of these approaches use a statistical parser to construct the syntactic representation S of the sentence and define a probability model that incorporates S. Chelba and Jelinek's **structured language model** [50] computes the joint probability of a word sequence and its parse S, $P(W, S)$ and decomposes it into a product of component probabilities involving various elements of the the word sequence proper, the head words from the parse structure, and the POS tags in the parse structure. Results reported in [50] indicate that a structured language model interpolated with a trigram model achieved a relative reduction in perplexity of 8% on the Wall Street Journal Continuous Speech Recognition (CSR) and Switchboard corpora. When used for lattice rescoring in a speech recognition system, it yielded a 6% relative reduction on the Wall Street Journal corpus and a 0.5% absolute reduction (41.1% to 40.6%) on Switchboard.

Another syntax-based model is the "almost-parsing" language model by Wang and Harper [51] (also called SuperARV model), which is based on a constraint-dependency grammar. Here, sentences are annotated with so-called SuperARVs, which are rich tags combining lexical features and syntactic information pertaining to a word (entry in the lexicon). A joint language model (the SuperARV language model) is defined over the word sequence and the sequence of tags as follows:

$$P(w_1,\ldots,w_N,t_1,\ldots,t_N) = \prod_{i=1}^{N} P(w_i t_i | w_1\ldots,w_{i-1},t_1\ldots t_{i-1}) \tag{5.34}$$

$$= \prod_{i=1}^{N} P(t_i | w_1\ldots,w_{i-1}) P(w_i | w_1\ldots w_{i-1},t_1\ldots t_{i-1}) \tag{5.35}$$

$$\approx \prod_{i=1}^{N} P(t_i | w_{i-2},w_{i-1},t_{i-1},t_{i-1}) P(w_i | w_{i-2},w_{i-1},t_{i-2},t_{i-1}) \tag{5.36}$$

The model is smoothed by recursive linear interpolation of higher-order and lower-order models. The SuperARV model was evaluated on the Wall Street Journal Penn Treebank and CSR tasks and was compared to other parser-based language models, including the structured language model described previously, as well as to standard trigram and POSbased models. It was found that the SuperARV achieved the lowest perplexity scores out of all. When used for lattice rescoring on the CSR task, the SuperARV model yielded relative word-error rate reductions between 3.1% and 13.5% and again outperformed all other models.

5.6.5. MaxEnt Language Models

One shortcoming of maximum-likelihood-based probability estimation for language models is that the constraints imposed by estimates solely derived from the training data are too strong. MaxEnt models represent an alternative where these constraints are relaxed. Rather than setting the probability of a given n-gram to its relative frequency in the training data (modulo smoothing), the requirement of MaxEnt modeling is that the model agree, on average, with the observed counts of events in the training data. The MaxEnt model is formulated as follows:

$$P(y|x) = \frac{1}{Z(x)} \exp\left(\sum_k \lambda_k f_k(x, y)\right) \tag{5.37}$$

where $f(x, y)$ is a feature function defined both on the input and the predicted variables, λ is a feature function specific weight, and $Z(x)$ is a normalization factor, computed as

$$Z(x) = \sum_{y \in Y} \exp\left(\sum_k \lambda_k f_k(x, y)\right) \tag{5.38}$$

Once appropriate feature functions have been defined, the expected value of f_k is

$$E(f_k) = \sum_{x \in X, y \in Y} \tilde{p}(x) p(y|x) f_k(x, y) \tag{5.39}$$

where $\tilde{p}(x)$ is the empirical distribution of x in the training data. The empirical expectation of f_k (derived from the training data) is

$$\tilde{E}(f_k) = \sum_{x \in X, y \in Y} \tilde{p}(x, y) f_k(x, y) \tag{5.40}$$

The model is then trained such that expected values match empirical expected values

$$E(f_k) = \tilde{E}(f_k) \qquad \forall k \tag{5.41}$$

while simultaneously maximizing the entropy of the distribution $p(y/x)$. This is equivalent to maximizing the conditional log-likelihood of the training data.

The MaxEnt framework was first applied to language modeling by Rosenfeld [52]. In the context of language modeling, y represents the predicted word and x the history or, more generally, the conditioning variables used for prediction. Note that in this case, a much larger context than the $n - 1$ directly preceding words may be included: feature functions may be defined over the entire sentence [53] or over an even larger domain. Most commonly, however, feature functions are simply defined over n-grams; for example, for a given word w_i and history h_i, a bigram feature function would be defined as follows:

$$f_{w_1, w_2}(h_i, w_i) = \begin{cases} 1 & \text{if } h_i \text{ ends in } w_1 \text{ and } w_i = w_2 \\ 0 & \text{otherwise} \end{cases} \tag{5.42}$$

The model can be trained using iterative methods, such as generalized iterative scaling [54] or improved iterative scaling [55], or by the faster quasi-Newton approaches (see [56]). Nevertheless, training of MaxEnt language models can be computationally demanding: in principle, the normalization factor in Equation 5.38 needs to be computed for all different values of x, and computing the feature expectations requires summation over all (x, y) pairs for which the feature is defined. Wu and Khudanpur [57] presented efficient training methods that result in considerable speedups during training. First the vocabulary is partitioned according to whether words are subject to

marginal or conditional constraints, and the summation required for the normalization factor is computed separately for both sets. Second, a hierarchical normalization computation procedure is proposed where partial sums (e.g., for histories ending in the same suffix) are reused. Together, these modifications led to speedups ranging between 15x and 30x.

Another potential problem of MaxEnt models is that they are prone to overfitting, especially when a large number of feature functions is used in relation to the number of samples. Possible solutions to this problem are feature selection [58], regularization [59], or incorporating priors on the feature functions. Using a Gaussian prior was proposed by Chen and Rosenfeld [59], for example. Rather than simply maximizing the conditional log-likelihood of the training data, argmax

$$\underset{\Lambda}{\operatorname{argmax}} \sum_{i=1}^{M} log P_{\Lambda}(y_i|x_i) \tag{5.43}$$

this approach maximizes the conditional log-likelihood modulo a penalty term composed of a product of zero-mean Gaussians for all feature functions:

$$\underset{\Lambda}{\operatorname{argmax}} \sum_{i=1}^{M} \log P_{\Lambda}(y_i|x_i) \times \prod_{k=1}^{K} \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(\frac{\lambda^2}{2\sigma_k^2}\right) \tag{5.44}$$

where σ_k^2 is the variance of the k^{th} Gaussian. Goodman [60] suggests exponential priors as an alternative, which can sometimes yield better performance.

Wu and Khudanpur [61] report on a MaxEnt model for speech recognition on the Switchboard task that incorporates topic constraints. The model achieved a 7% perplexity reduction and an absolute word-error rate reduction of 0.7% (38.5% to 37.8%). Integration of syntactic constraints independently yielded a 7% perplexity reduction and a 0.8% reduction in word-error rate. The combination of both types of constraints was shown to be additive, yielding a 12% relative perplexity reduction and an absolute word-error rate improvement of 1.3%.

5.6.6. Factored Language Models

The factored language model (FLM) approach [62, 63] builds on the observations that the prediction of words is dependent on the surface form of the preceding words and that better generalizations can be made when additional information, such as the word's POS or morphological class, is taken into account. In particular, word n-gram counts might be insufficient to robustly estimate the probability of word w_i given word w_{i-1}, but if we know that word w_{i-1} is of a particular class, say a determiner, we might be able to obtain a good probability estimate for P (w_i| determiner). This is reminiscent of the class-based models described previously; however, in an FLM, many such class-based estimates are combined and structured hierarchically through the use of a **generalized backoff** strategy. FLMs assume a factored word representation, where words are considered feature vectors rather than individual surface forms; that is, $W \equiv f_{1:K}$. An example is shown here:

WORD:	*Stock*	*prices*	*are*	*rising*
STEM:	Stock	price	be	rise
TAG:	Nsg	N3pl	V3pl	Vpart

The word surface form itself can be one of the features. A statistical model over this representation can be defined as follows (using a trigram approximation):

$$p\left(f_1^{1:K}, f_2^{1:K}, \ldots, f_t^{1:K}\right) \approx \prod_{i=3}^{t} p\left(f_i^{1:K} | f_{i-1}^{1:K}, \ldots, f_{i-2}^{1:K}\right) \tag{5.45}$$

Thus, each word is dependent not only on a single stream of temporally ordered word variables but also on additional parallel (i.e., simultaneously occurring) feature variables.

In the definition of standard backoff (Equation 5.8), the model backs off from the higherorder to the next lower-order distribution. In an FLM, however, it is not immediately obvious how backoff should proceed because conditioning variables are not only temporally ordered but also occur in parallel. Thus, a decision must be made as to which subset

of features the model should back off to in which order. In principle, there are several different ways of choosing among different backoff "paths":

1. Choose a fixed, predetermined backoff path based on linguistic knowledge (e.g., always drop syntactic before morphological variables).
2. Choose the path at runtime based on statistical criteria.
3. Choose multiple paths and combine their probability estimates.

The last option, called **parallel backoff**, is implemented via a new, generalized backoff function (here shown for a trigram):

$$P_{GBO}(f|f_1,f_2) = \begin{cases} d_c P_{ML}(f|f_1,f_2) \text{ if } c > \tau \\ \alpha(f_1,f_2) g(f,f_1,f_2) \text{ otherwise} \end{cases} \quad (5.46)$$

where, similar to Equation 5.8, c is the count of (f, f_1, f_2), $P_{ML}(f|f_1, f_2)$ is the maximum likelihood estimate, τ is the count threshold, and $\alpha(f_1, f_2)$ is the normalization factor that ensures that the resulting scores form a probability distribution. The function $g(f, f_1, f_2)$ determines the backoff strategy. In a typical backoff procedure, $g(f, f_1, f_2)$ equals $P_{BO}(f|f_1)$. In generalized parallel backoff, however, g can be any nonnegative function of f, f_1, f_2 and can be instantiated to, for example, the mean, weighted mean, product, or maximum functions. For example, the mean function would take the average of the individual estimates:

$$g_{\text{mean}}(f, f_1, f2) = 0.5 P_{BO}(f|f_1) + 0.5 P_{BO}(f|f_2) \quad (5.47)$$

In addition to different choices for g, different discounting parameters can be chosen at different levels in the backoff graph.

It is not a priori obvious which backoff strategy works best; the optimal strategy is highly dependent on the particular language modeling task. Because the space of possible factored language model structures and backoff parameters is very large, it is advisable to use an automatic, data-driven procedure to find the best settings. An automatic FLM

optimization procedure based on genetic algorithms was proposed by Duh and Kirchhoff [64].

FLMs have been implemented as an add-on to the widely used SRILM (Stanford Research Institute Language Modeling) toolkit [65] and have been used successfully for morpheme-based language modeling [62], multispeaker language modeling [66], dialog act tagging [67], and speech recognition [68, 63], especially in sparse data scenarios such as highly inflecting languages (see also §5.7.2).

5.6.7. Other Tree-Based Language Models

Several other language modeling approaches make use of tree structures; one, for example, is the hierarchical class-based backoff model proposed by Zitouni [69]. Here, backoff proceeds along a hierarchy of word classes arranged in a tree structure, with more general classes at the top and more specific classes at the bottom. Backoff proceeds along the class hierarchy from bottom to top; that is, more specific backoff classes are utilized before more general ones. The main differences from FLMs are that the backoff path is fixed and predefined in advance, whereas FLMs permit the combination of probability estimates from different paths in the backoff graph, as well as the dynamic choice of a path at runtime. Zitouni found that a hierarchical class-based language model yields gains primarily when the test set contains a large number of unseen events: on a speech recognition language modeling task with a 5,000-word vocabulary, the unseen word perplexity was reduced by 10%, whereas on a task with a 20,000-word vocabulary, it was reduced by 26% and the word-error rate decreased by 12%. Some extensions to this hierarchical class n-gram language model were proposed by Wang and Vergyri [70]. Specifically, POS information was integrated into the word clustering process, and separate hierarchical class trees were defined for different POS categories. On an Egyptian Colloquial Arabic speech recognition task (a test set of 18,000

words), the model achieved 8% relative improvement in perplexity over a standard n-gram model and a 3% relative improvement over the model described by Zitouni [69].

Random forest language models (RFLMs), proposed by Xu and Jelinek [71], model all word histories seen in the training data as a collection of randomly grown decision trees (a random forest). Nodes in the decision trees are associated with sets of histories, with the root node containing all histories. Trees are grown by dividing the set of histories into two subsets according to the word identity at a particular position in the history. Out of a number of possible splits, the split that maximizes the log-likelihood of the training data is chosen. Two steps are taken to introduce randomness into this process: first, the set of histories from a parent node are initially assigned randomly to the two subnodes; second, the set of splits chosen to undergo the log-likelihood test is selected randomly as well. After the growing process has stopped, each leaf node in a decision tree can be viewed as a cluster of similar word histories forming an equivalence class. Rather than defining equivalence classes deterministically by the most recent $n - 1$ words (as done by conventional n-gram models), equivalence classes are now defined based on the set membership of the words in the history.

The decision tree–growing procedure is run multiple times, and the decision trees resulting from each run are added to the random forest. Supposing that M decision trees have been obtained, the RFLM probabilities are computed as averages of the individual decision tree probabilities:

$$P_{RF}(w_i|w_{i-n+1},\dots w_{i-1}) = \frac{1}{M}\sum_{j=1}^{M} P_{DT_j}(w_i|\phi_{DT_j}(w_{i-n+1},\dots,w_{i-1})) \tag{5.48}$$

Here, φ_{DT_j} is the *j'th* is a function mapping the history $w_{i-1+n+1},\dots, w_{i-1}$ to a leaf node in the j'th decision tree. The number of decision trees M usually ranges in the dozens or hundreds. Perplexity and word-error rate tests on the Penn Treebank portion of the Wall Street Journal

corpus showed that the perplexity of a random forest trigram was reduced by 10.6% relative to a trigram with interpolated Kneser-Ney smoothing. Interpolation of the RFLM with a Kneser-Ney model did not improve perplexity any further. N -best list rescoring with RFLMs yielded a relative word-error rate improvement of 11% on the Wall Street Journal DARPA'93 HUB1 benchmark task. Since their inception, RFLMs have been applied to structured language modeling [71] and prosodic modeling [72]. In the context of multilingual language modeling, they have also been applied to morphologically rich languages (see §5.7.1).

5.6.8. Bayesian Topic-Based Language Models

A significant recent trend in statistical language modeling is Bayesian modeling of latent topic structure in documents. One of the first models of this type was the latent Dirichlet allocation model proposed by Blei, Ng, and Jordan [25]. The LDA model assumes that a document is composed of K topics, denoted as $z_1,\ldots, z_K$. Each topic generates words according to a topic-specific distribution over individual words (i.e., a topic is modeled as a bag of words; n-grams are not taken into account). The probability vector over words is denoted by φ_k for each topic $k = 1,\ldots,K$. Each topic has a prior probability, denoted by θ_k. The topic priors $\theta_1,\theta_2,\ldots,\theta_K$ are themselves distributed according to the Dirichlet distribution with hyperparameters $\alpha_1,\ldots,\alpha_K$ (see §5.4.2 for an explanation of the Dirichlet distribution):

$$P(\theta_1,\ldots,\theta_k) = \frac{\Gamma(\sum_k \alpha_k)}{\prod_k \Gamma(\alpha_k)} \prod_{k=1}^{K} \theta_k^{\alpha_k - 1} \tag{5.49}$$

The generative model underlying this approach is that a set of priors $\theta_1,\ldots,\theta_K$ is sampled from the Dirichlet distribution. A given topic z_k is chosen with probability θ_k, then a word w is generated from the topic with probability $\varphi_k(w)$. The probability of an entire document consisting of a sequence W of t words is modeled as

$$p(W|\alpha,\phi) = \int p(\theta|\alpha)\left(\prod_{i=1}^{t}\sum_{z_i} p(z_i|\theta)p(w_i|z_i,\phi)\right)d\theta \quad (5.50)$$

The challenging aspect of LDA is computing the posterior distribution of the latent variables θ and $\mathbf{z}$, $p(\theta, \mathbf{z}|W, \alpha, \varphi)$, which is not amenable to exact inference. Sampling techniques such as Markov chain Monte Carlo (e.g., [73]) or variational inference [25] need to be used.

Because the LDA model is a unigram model, it also needs to be combined with an n-gram model for practical purposes. Wang et al. [74] combined LDA with a trigram and a probabilistic context-free grammar, yielding perplexity reductions of 9% to 23% on the Wall Street Journal corpus relative to a Kneser-Ney smoothed trigram model. Hsu and Glass [75] used LDA combined with a hidden Markov model for a spoken lecture recognition task. A combination of language models trained with the topic labels provided by this model yielded a 16.1% perplexity reduction and a 2.4% word-error rate reduction over an already adapted trigram model.

The LDA model has been extended in various ways: first, LDA can be generalized to utilize Dirichlet processes [76], a prior for nonparametric models that can handle an infinite number of topics. Thus, rather than assuming a fixed set of K topics, the number can be adjusted on the basis of training data properties. Second, the latent topic variables can be structured hierarchically: each topic can have a number of subtopics, and individual topics can be shared between different data clusters. This is modeled by a hierarchical Dirichlet process (HDP) [77]. Huang and Renals exploited HDPs for integrating topic and participant role into language models for meeting-style conversational speech recognition. HDP-adapted language models yielded slight word-error-rate reductions (0.3% absolute) compared to standard adaptation methods, for baseline systems ranging around 39% worderror rate. Teh [78] reports on a Bayesian language model based on a Pitman-Yor process that by itself achieves

perplexities comparable to those of a Kneser-Ney smoothed trigram model (without requiring interpolation with the baseline model).

5.6.9. Neural Network Language Models

With the exception of LSA-based language models, all of the language modeling approaches described previously estimate probabilities for events in a discrete space. Neural network language models (NNLMs) [79] take a different approach: discrete word sequences are first mapped into a continuous representation, and n-gram probabilities are then estimated within this continuous space. The assumption is that words having similar distributional properties will receive similar continuous representations, which will in turn result in smoother probability estimates.

The neural network is typically a multilayer perceptron with an input layer, a projection layer, a hidden layer, and an output layer of nodes. A graphical representation of the architecture of an NNLM is shown in Figure 5–1. Adjacent layers are fully interconnected by weights. For a vocabulary of V words, the input consists of a concatenation of $n - 1$ V-dimensional binary feature vectors representing the history of $n - 1$ words (e.g., the first two words in a trigram). The projection layer i of a given fixed dimensionality d encodes the shared continuous representation of the words, which is learned during training. The hidden layer h also has a fixed number of J nodes, each of which computes a thresholded, nonlinear combination of incoming activations, for example, using the tangent function:

$$h_j = \tanh\left(\sum_{k=1}^{d} w_{jk}^h i_k + b_j^h\right) \qquad \forall j, i = 1, \ldots, J \tag{5.51}$$

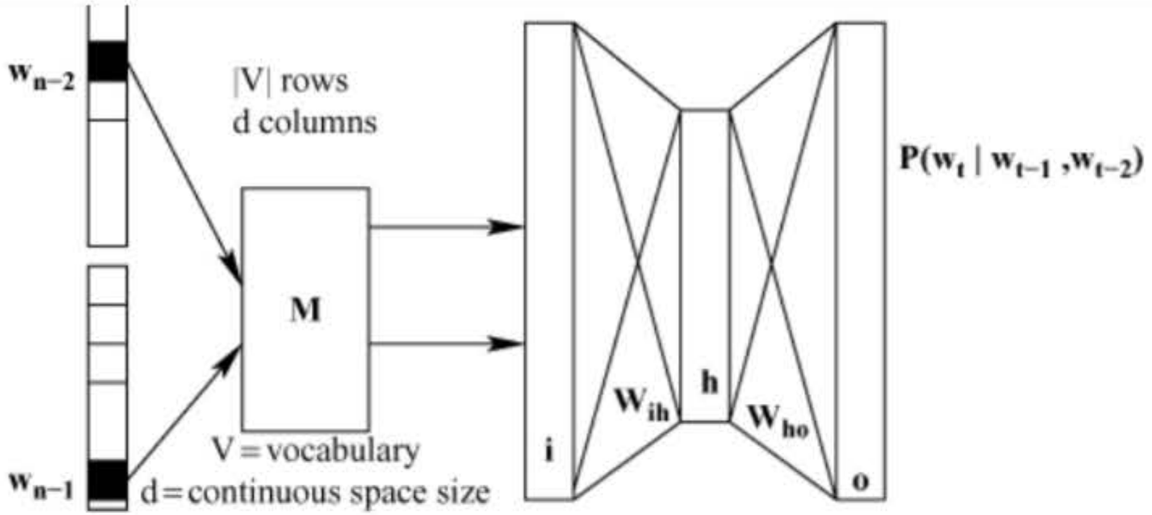

Figure 5–1. Neural network language model

where w^h denotes the weights connecting the projection, and the hidden layers and b^h are the biases on the hidden layer nodes. Finally, the output layer o computes a posterior probability distribution over V by

$$o_j = \frac{a_j}{\sum_{k=1}^{V} a_k} \qquad \forall j, j = 1, \ldots, V \tag{5.52}$$

and

$$a_j = \sum_{k=1}^{J} w_{jk}^o h_k + b_j^o \tag{5.53}$$

During training, binary target labels are associated with the output layer, 1 for the predicted word and 0 for all other words. The network is trained (e.g., using backpropagation) to maximize the log-likelihood of the training data, possibly enriched with a regularization term R to constrain the parameter values θ:

$$L = \frac{1}{T} \sum_{i=1}^{T} log(P(w_i|h_i); \theta) - R(\theta) \tag{5.54}$$

The regularization term can take various forms; a common approach is to use the sum of the squared weights: $\Sigma_w w^2$. This limits the complexity of the network and reduces the chance of overfitting by preventing the weights from becoming too large. Thus, a particular word in the history is first projected into a continuous representation shared among all words, which is then used as the basis for estimating the probability of the predicted word given its history.

NNLMs have been successfully used for speech recognition by Schwenk and Gauvain [80], Schwenk [81], and Schwenk, Déchelotte, and Gauvain [82], for example, and for machine translation by Alexandrescu and Kirchhoff [83]. Although they typically use a truncated vocabulary consisting of the m most frequent words only, they have yielded significant improvements when used in combination with standard n-gram models. Schwenk [81] reported an 8% relative reduction in perplexity and a 0.5% absolute improvement in word-error rate on a French broadcast news recognition task. Emami and Mangu [82] obtained a 0.8% absolute (3.8% relative) improvement in word-error rate on an Arabic speech recognition task.

Emami and Jelinek [84] used neural network–based probability estimation in combination with a structured language model; Alexandrescu and Kirchhoff [85] combined NNLMs with a factored word representation for Arabic to be able to exploit word similarities derived not only from distributional properties but from morphological and POS classes.

5.7. Language-Specific Modeling Problems

The majority of language modeling research has focused on the English language. However, speech and language processing technology has been ported to a range of other languages, some of which have highlighted problems with the standard n-gram modeling approach and have necessitated modifications to the traditional language modeling framework. In this section, we look at three types of language-specific problems: morphological complexity, lack of word segmentation, and spoken versus written languages.

5.7.1. Language Modeling for Morphologically Rich Languages

A morphologically rich language is characterized by a large

number of different unique word forms (types) in relation to the number of word tokens in a text that is due to productive morphological (word-formation) processes in the language. A morpheme is the smallest meaning-bearing unit in a language. Morphemes can be either free (i.e., they can occur on their own), or they are bound (i.e., they must be combined with some other morpheme). Morphological processes include compounding (forming a new word out of two independently existing free morphemes), derivation (combination of a free morpheme and a bound morpheme to form a new word), and inflection (combination of a free and a bound morpheme to signal a particular grammatical feature).

Germanic languages, for example, are notorious for their high degree of compounding, especially for nominals. Turkish, an agglutinative language, combines several morphemes into a single word; thus, the same material that would be expressed as a syntactic phrase in English can be found as a single whitespace-delimited unit in a Turkish sentence, such as: *görülmemeliydik* = 'we should not have been seen'.

As a result, Turkish has a huge number of possible words. Many languages have rich inflectional paradigms. In languages like Finnish and Arabic, a root (base form) may have thousands of different morphological realizations. Table 5–1 shows two Modern Standard Arabic (MSA) inflectional paradigms, one for present tense verbal inflections for the root *skn* (basic meaning: 'live'), one for pronominal possessive inflections for the root *ktb* (basic meaning 'book').

Table 5–1. MSA inflectional paradigms for present tense verb forms and possessive pronouns (affixes are separated from the word stem by hyphens)

Word	Meaning	Word	Meaning
'a-skun(u)	I live	kitaab-iy	my book
ta-skun(u)	you (MASC) live	kitaabu-ka	your (MASC) book
ta-skun-iyna	you (FEM) live	kitaabu-ki	your (FEM) book
ya-skun(u)	he lives	kitaabu-hu	his book
ta-skun(u)	she lives	kitaabu-haa	her book
na-skun(u)	we live	kitaaabu-na	our book
ta-skun-uwna	you (MASC-PL) live	kitaabu-kum	your book
ya-skun-uwna	they live	kitaabu-hum	their book

Morphological complexity causes problems for language modeling due to the high ratio of types to tokens, which results in a lack of training data: many n-grams in the test data are not seen at all in the training data or are not observed frequently enough, leading to unreliable probability estimates. Another effect is a high OOV rate. To some extent, this effect can be mitigated by simply collecting more training data; however, depending on the morphological complexity of the language, the vocabulary growth does not slow down as sharply as for morphologically poor languages as more and more running text is taken into account. Table 5–2 shows examples of the relationship between word types and word tokens for different languages, as well as typical OOV rates for different languages on held-out test sets.

Table 5–2. Number of word tokens, word types, and OOV rates for different languages in non-decomposed form

Language	Style	# Tokens	# Types	OOV Rate at (N) Words	Source
English	news text	19M	105k	1% (60k)	[86]
Arabic	news text	19M	690k	11% (60k)	[86]
Czech	news text	16M	415k	8% (60k)	[87]
Korean	news text	15.5M	1.5M	25% (100k)	[88]
Turkish	mixed text	9M	460k	12% (460k)	[89]
Finnish	news text, books	150M	4M	1.5% (4M)	[90]

When processing morphologically rich languages, it must be determined whether the vocabulary needed for a given application can be expressed as a list of full word forms

(e.g., the most frequent forms occurring in the training data) or whether smaller subword units need to be chosen as basic language modeling units. The choice depends on the constraints imposed by available computing resources, such as the efficiency of the decoder in a speech recognition application, memory, and desired speed, as well as on the amount of training data. The advantage of decomposing words into smaller units lies in the reduction of the vocabulary size, which in turn reduces the number of distinct n-grams. In addition to improving speed and reducing memory consumption, subword units occur in multiple words; therefore, training data can be shared across words, and the number of training tokens per unit increases, which leads to more robust probability estimation. Finally, modeling based on subword units might also allow the language model to assign probabilities to words that were not seen in the training data. On the other hand, if a word is decomposed linearly, a fixed n-gram context provides only information about the relationship between different parts of a word but not about interword dependencies. Thus, the predictive power of the language model is diminished. Moreover, when the language modeling vocabulary is identical to the vocabulary used in a speech recognizer, care must be taken to define units that do not become too short and hence acoustically confusable.

Arabic, generally recognized as a morphologically rich language, is an interesting example highlighting different situations where decomposition may or may not be required. Several studies [68, 63, 91] have shown that integrating morphological information into the language model is helpful for modeling dialectal Arabic. Although the dialects of Arabic exhibit morphological simplifications compared to MSA (the written standard), they have very sparse training data because they are essentially spoken languages and need to be transcribed manually to obtain language modeling data. Large amounts of data are available for

MSA, and significant improvements through morphological decomposition in the language model have not been observed for this variety when large training sets were used [91]. Moreover, the vocabulary size needed for large-scale applications such as speech recognition of MSA is around 600,000 to 800,000 words, which is within the range that current decoders can accommodate [92].

It is obvious, however, that for languages with particularly high type to token ratios (e.g., Finnish or Turkish), some form of decomposition is required: for large tasks, the size of the vocabulary needed to achieve sufficient coverage of the test data is likely to exceed current decoder capabilities, and the corresponding language model would not be able to be estimated reliably. In the following section, we discuss several recent approaches to the problem of word decomposition.

5.7.2. Selection of Subword Units

The identification of subword units can be performed in a data-driven, unsupervised way; it can be based on linguistic information (e.g., a morphological analyzer); or it can be a combination of both. Linguistically based methods typically involve a handcrafted morphological analysis tool, such as the Buckwalter Morphological Analyzer developed for Arabic [93], which analyzes each word form into its morphological components. In the case where several possible analyses are provided for each word form, statistical disambiguation needs to be performed in a subsequent step (e.g., [94]). Data-driven approaches may incorporate varying degrees of information about the specific language in question, and they may vary with respect to the optimization criterion. Some approaches are intended to discover units corresponding to linguistically defined morphemes, whereas others are aimed at selecting an inventory of units that is best suited to the task or application at hand.

Automatic algorithms for identifying linguistic morphemes are as old as Zellig Harris's 1955 approach of estimating the perplexity of different letters following each letter in a word [95]. If the perplexity at a certain transition is high (i.e., the following letters are not easily predictable) a morpheme boundary can be hypothesized at that point. Adda-Decker and Lamel [96] show how a modified version of this approach can be used to decompose German compounds, thus reducing the OOV rate by a relative amount of 23% to 50%, in a German corpus of 300 million words and a fixed vocabulary ranging from 65,000 to 100,000.

In general, simple frequency-based approaches are prone to overfitting to the training data and hypothesizing more morphemes than desired, because the fit to the data is not balanced against the total size of the inventory. This shortcoming is addressed by models that include explicit penalty terms for the size of the morpheme inventory. The more recently developed Morfessor package [97] is one such example. It attempts to derive a morpheme inventory M from a corpus C by maximizing its posterior probability:

$$M = \underset{M}{\operatorname{argmax}}\, P(M|C) = P(C|M)P(M) \tag{5.55}$$

which is equivalent to a minimum description length approach. The search over possible morphemes proceeds by way of a greedy algorithm that recursively splits each word into two subparts, trying out all possible positions. Those splits that improve the probability $P(M/C)$ (reduce the code length) are chosen. Later versions of this approach [98, 99] include a stochastic morphological category model and different probability estimation techniques. Morfessor models outperformed most other automatic segmentation algorithms in a benchmark evaluations task involving Finnish, Turkish, and English [100]. Language models using Morfessor to decompose words have been applied to Finnish, Estonian, Turkish, and Arabic speech recognition and achieved good results on the first three languages, which are highly agglutinative [90].

An alternative to trying to match a predefined linguistic inventory is to derive a unit inventory that directly optimizes a criterion related to language model performance, such as perplexity or OOV rate. This may be preferable in cases where languages are not strictly agglutinative but contain some amount of fusion, such as nontransparent changes in the word form produced by the combination of two or more morphemes. This approach has been pursued by Whittaker and Woodland [101], for example, for Russian language modeling. Here, a particle-based model was developed, defined as

$$P(w_i|h) = \frac{1}{Z(h)} P\left(u_{L(w_i)}^{w_i} | u_{L(w_i)-1}^{w_i}\right) P\left(u_{L(w_i)-1}^{w_i} | u_{L(w_i)-2}^{w_i}\right), \ldots, P(u_1^{w_i}) \left| u_{L(w_{i-1})}^{w_{i-1}}\right) \quad (5.56)$$

where word w_i is decomposed by some decomposition function L into $L(w_i)$ particles u_1,..., $u_L(w_i)$. The particle language model then computes the probability of a particle given its history consisting of all particles up to the last particle in the preceding word. Two data-driven methods for deriving particles were compared: a greedy search over possible units of a fixed length, retaining those particles that maximizing the likelihood of the data, and a particle-growing technique whereby particles are initialized to all single-character units and are successively extended by adding surrounding characters such that the resulting units minimize perplexity.

Kiecza, Schultz, and Waibel [88] proposed an approach to Korean language modeling whereby elementary syllable units are combined into units larger than syllables but smaller than Korean words (called *eojols*), which are of a complexity similar to Turkish words. Syllable combination was done by minimizing OOV rate. In both these approaches, significant improvements in perplexity/OOV rate, but only limited gains if any in the final system evaluation (speech recognition word-error rate), were reported.

More recently, the choice of subword units has been optimized with respect to the final system performance, usually by trying different segmentation schemes and

evaluating each with respect to its effect on the performance metric. An example is presented by Arisoy, Sak, and Saraçlar [102], where words, statistically derived units, linguistically defined morphemes, and stems plus endings were compared for the purpose of Turkish language modeling for speech recognition, with the result that stems plus endings yielded the best word-error rate out of all four choices.

5.7.3. Modeling with Morphological Categories

Most of the work on language modeling with subword units has focused on linear decomposition of words, primarily for agglutinative languages. As a consequence, the resulting subword units are most frequently used in a standard n-gram model. As mentioned earlier, one problem is that the n-gram context needs to be increased in order to be able to model interword dependencies in addition to dependencies between subword units. This in turn places demands on the amount of training data needed.

However, several alternative approaches have been developed wherein the word remains the basic modeling unit but the probability assignment takes into account statistics over subword components or morphological classes. Arisoy et al. [46] proposed a discriminative language model (see §5.6.3) for Turkish that incorporates feature functions defined over morphemes, such as root n-gram counts or inflectional group counts. The language model assigns probabilities to sequences of entire words (undecomposed n-best hypotheses), but does so by taking into account the constraints provided by morpheme-based feature functions. This model was shown to provide slightly better results (0.3% absolute word-error rate reduction on a Broadcast News recognition task) than a discriminative language model using only word-based features. The same approach was taken by Shafran and Hall [45] for Czech, with similar results.

Kirchhoff et al. [63] and Vergyri et al. [68] used both morphological classes (stem, root, etc.) and words as conditioning variables in a factored language model for Arabic. Although the model predicts probabilities for entire word forms, the probabilistic backoff procedure makes use of morphological constituents. On a dialectal Arabic speech recognition task with a limited amount of training data, FLMs yielded slight reductions in word error rate (0.5%–1.5% absolute). FLMs have also been used successfully for speech recognition of other highly inflected languages, such as Estonian [103], and for machine translation [104].

Morphological features were also used as additional input features (beyond words) to a neural language model (see §5.6.9) for both Arabic and Turkish [85], thus creating a factored neural language model. Perplexity was shown to improve substantially over a wordbased neural language model (up to 10% for Arabic and 40% for Turkish), but no application results were reported.

Sarikaya and Deng [105] proposed a joint morphological-lexical language model for Arabic. Here, a sentence is annotated with a rich parse tree representing morphological, syntactic, semantic, and other attribute information. The language model predicts the probability of the word string and the associated tree jointly, using MaxEnt probability estimation (see §5.6.5). The model was evaluated on an English-to-Arabic translation task and improved the BLEU score by 0.3 points (absolute) over a word trigram model and 0.6 points over a morpheme trigram model.

RFLMs (see §5.6.7) were applied to morphological language modeling by Oparin et al. [106]. The difference from standard word-based RFLMs is that the decision trees used in the random forest model can ask questions not only about the set membership of words but also about morphological features (inflections or morphological tags), stems, lemmas, and parts of speech. Morphological RFLMs were evaluated on a Czech spoken lecture recognition task with a 240,000-

word vocabulary. Whereas word-based RFLMs did not show any substantial gain over Kneser-Ney-style trigram language models, morphological RFLMs yielded a relative gain in perplexity of up to 10.4% and a relative improvement in word accuracy of up to 3.4%. Unlike previous studies' results, it was also found that interpolation of RFLMs and standard *n*-gram models yielded an improvement (of up to 15.6% in perplexity). In addition to producing different word history clusters (induced by morphological features rather than words), a morphological RFLM offers more possibilities for randomization because the set of potential splits at each decision tree node is greatly enlarged, which may contribute to the superior performance of morphological RFLMs in this case.

5.7.4. Languages without Word Segmentation

Although agglutination produces a large number of long and complex word forms in many languages, other languages do not possess any explicit segmentation of character strings into words at all. In languages such as Chinese and Japanese, sentences are written as sequences of characters delimited by punctuation signs but without any intervening whitespaces to indicate word boundaries. Readers with knowledge of the language can immediately decompose character sequences into the word segmentation that corresponds to the most plausible interpretation of the sentence. Although statistical language models for such languages can be constructed over characters, it is preferable to first segment sentences into words automatically and then train the language model over the resulting words. Similar to the situation in which words are decomposed into subword units (§5.7.1), a language model using characters as the basic modeling units may fail to express important interword relationships. Moreover, the word segmentation may determine how characters are pronounced, which is important if the same modeling units are intended to be

used in the language model and the acoustic model of a speech recognition system. Finally, it has been shown experimentally that a language model built on automatically segmented text outperforms a character-based language model in terms of perplexity for both Japanese [107] and Chinese [108].

Automatic segmentation algorithms typically use a combination of dictionary information, statistical search, and additional features such as nonnative letters, character co-occurrence counts, and character positions. Generating the most likely segmentation follows a statistical decoding framework that uses, for example, Viterbi search. Other modeling approaches that have been explored recently include conditional random fields [109, 110, 111], MaxEnt modeling [112, 113], and discriminative modeling using perceptrons [114]. Much of the work on Chinese word segmentation has been performed in the context of the SIGHAN Chinese word segmentation competitions that have been taking place since 2003, organized by the Association for Computational Linguistics. This competition serves as a benchmark comparing different word segmentation systems. Automatic word segmentations are compared against a linguistically segmented gold standard and are evaluated in terms of precision (P), that is, the percentage of correct cases out of all hypothesized boundaries, recall R (the percentage of identified boundaries out of all possible boundaries), and their combination in the form of F-measure, $F = 2PR/(P + R)$. These measures can be calculated separately on the OOV words versus in-vocabulary words. The best-performing systems in the most recent evaluation achieved an F-measure of 0.96; however, performance on OOV words was much lower with F-measures around 0.76 [115].

Rather than trying to match linguistically defined words, it is also possible to optimize segmentation directly for language model performance. Sproat et al. [108] showed

that the dictionary used for Chinese word segmentation significantly influences the perplexity of a bigram language model trained on the segmented text and that it is possible to iteratively optimize the dictionary by merging frequent word co-occurrences, such that the perplexity is reduced at each iteration. Note that such approaches are similar to the data-driven algorithms for deriving morpheme-like subword units described earlier in Section 5.7.1. Another example of a data-driven approach, in this case for Japanese, uses chunks of characters for language modeling [107]. Chunks are derived from the training data by selecting the highest-frequency n-grams and additional patterns with close similarity to them. Thus, the basic modeling units are neither characters nor words but intermediate units.

5.7.5. Spoken versus Written Languages

Statistical language modeling crucially relies on large amounts of written text data, and a significant trend within language modeling research is the development of methods for scaling current language modeling techniques to ever-larger databases. However, many of the world's 6,900 languages are spoken languages, that is, languages without a writing system. They are either indigenous languages without a literary tradition, or they are linguistic varieties such as regional dialects that are used in everyday spoken communication but are rarely put into writing. This is the case, for example, with the many dialects of Arabic, which are used in everyday conversation but are almost never found in written form. Other languages may be spoken as well as written but may not have a standardized orthography.

Both cases represent difficulties for language modeling. In the first case, the only way of obtaining language model training data is to manually transcribe the language or dialect. This is a costly and time-consuming process because it involves (i) the development of a writing standard,

(ii) training native speakers to use the writing system consistently and accurately, and (iii) the actual transcription effort. In the second case, those text resources that can be obtained for the language in question (e.g., from the web) will need to be normalized, which can also be a laborious process. As a consequence, very little work has been carried out on language model development for such underresourced languages. Most studies have concentrated on how to rapidly collect corpora for underresourced languages using web resources. Some of the challenges inherent in this process are described by Le et al. [116] and Ghani, Jones, and Mladenic [117]. Possible methods for rapid language model bootstrapping for spoken languages and languages characterized by a lack of standardization might include grammar-based or class-based approaches in combination with a limited amount of transcribed material. For a constrained application, such as the development of a dialog system, the structure of possible utterances could be predefined by a task grammar or a class-based language model, whereas more fine-grained word sequence probabilities, or probabilities of words given their classes, are modeled by a language model trained from a small amount of data. An interesting research direction is the possible use of data from language that is closely related to the language in question or from a language that is unrelated but resource rich. The following section describes some of these approaches.

5.8. Multilingual and Crosslingual Language Modeling

5.8.1. Multilingual Language Modeling

Up to this point we have discussed problems that arise when tailoring statistical language models to a particular language or language type, such as agglutinative languages or languages without word segmentation. The tacit assumption has been that the resulting language model is

used in an application where only the language of interest is encountered. However, in many situations, a system can be presented with multiple languages sequentially (e.g., different users speaking different languages, without advance indication of which language will be encountered next), or simultaneously, as happens in the case of **code switching**. Here, speakers may use several languages or dialects side by side, often within the same utterance. The phenomenon of code switching exists in a variety of bilingual and multilingual communities or where diglossia exists, such as where a formal standard language is used in addition to colloquial or dialectal varieties. The use of "Spanglish" (mixed Spanish/English) in the United States is one example of code switching, as demonstrated by the following example from Franco and Solorio [118]:

I need to tell her que no voy a poder ir.
'I need to tell her that I won't be able to make it.'

To handle multilingual input where language can be switched dynamically between utterances, separate language models can be constructed from monolingual corpora, and a system using these models (e.g., a speech-based information kiosk or telephone-based dialog system) can access them dynamically based on the output from a first-step language identification module or based on which language model (possibly combined with an acoustic model in the case of speech recognition) yields the highest scores after the first processing steps.

Fügen et al. showed how several such monolingual models can be combined into a single multilingual language model by means of a context-free grammar whose nonterminal states can encode language information and whose terminal states correspond to monolingual n-gram models. Explicit grammar rules can be leveraged to expand current states with n-grams from the matching language only, thus preventing language switching at inappropriate times. Alternative ways of constructing a single multilingual

language model are to combine monolingual corpora and train a single model on the pooled data or to interpolate several monolingual language models. The first technique has been shown to degrade performance, especially when the sizes of the corpora are unbalanced [120, 121]. The second technique may fare slightly better yet was shown to be inferior to the grammar-based combination method described earlier [119].

Building a language model for the second case (intrasentential language switching) is more difficult because little or no relevant training material is available. Weng et al. [122] built a four-lingual language model by introducing a common backoff node in the form of a pause unit; that is, language switches are allowed with some probability after a pause has occurred.

5.8.2. Crosslingual Language Modeling

Another question that might be asked is whether data in one language can be leveraged to improve a language model for a different language, provided that the style or domain are closely related. If insufficient data is available for the language of interest, inaccurate probability estimation might be improved if sufficient information can be extracted from a large amount of foreign-language text.

The most straightforward way of applying this idea is to automatically translate the foreign-language text into the desired language and to use the translated text (though errorful) as additional language training data. This approach was chosen, for example, by Khudanpur and Kim [123] and Jensson et al. [124].

In the former study, training data for a Mandarin news text language model intended for speech recognition was enriched with training data obtained by automatically translating English text from the same domain. A unigram extracted from the translated text was interpolated with a trigram baseline language model trained on the available

Mandarin data. The selection of English text for translation, and the determination of the interpolation parameter λ were specific to each news story, thus providing at the same time an implicit form of topic adaptation. The resulting model improved character perplexity by about 10% relative and the word-error rate of the speech recognizer by 0.5% absolute (for various different systems ranging around 26% baseline character-error rate). The authors also noted that the English text was more recent than the Chinese text, which may have contributed to the improved performance. This is an important consideration when investigating potential out-of-language data resources.

Jensson et al. [124] developed a language model for weather reports in Icelandic using a small amount of in-language data and a limited amount of parallel English–Icelandic data for training a machine translation system. A language model trained on a larger set of automatically translated data was then interpolated with the baseline language model, with positive effects on perplexity (9.20% improvement) and word-error rate (1.9–9.5% relative improvement) of an Icelandic speech recognition system.

The use of machine translation technology for processing out-of-language data is likely to fail when the desired language does not have sufficient data to train a machine translation system in the first place, although the abovementioned experiments on Icelandic show that a limited amount of parallel data may still be sufficient if the domain is very constrained. An alternative to using a fully fledged machine translation system is to rely on a high-quality word-based translation dictionary only. Kim and Khudanpur [125] showed that a goodquality translation dictionary can be extracted by simply computing mutual information statistics on word pairs from document-aligned parallel corpora, eliminating the need for sentence-aligned parallel text. In their experiments on Mandarin broadcast news recognition, they found that a unigram

model built from the resulting dictionary-based translations and interpolated with a baseline language model achieved a performance similar to that of a crosslingual language model. Another possibility for constructing a translation dictionary from document-aligned data is to use crosslingual latent semantic analysis [126]. Under this approach, words in both languages are projected into a common semantic space, and a measure of similarity between words from different languages in this space is used to construct word translation probabilities.

A drawback of the previous approaches is that the quality of the resulting model is heavily dependent on the translation accuracy. Tam et al. [127] recently presented an alternative model in which bilingual latent semantic analysis (bLSA) is used for adaptation before translation. The approach uses one LSA model each for the source and the target languages; it thus requires a parallel training corpus. The LSA models incorporate Dirichlet-style prior distributions over topics (see §5.6.8). Topic mixture weights are determined from the source LSA model and are projected onto the target LSA model, where they can be used to compute target-language marginal distributions. Assuming that the source language is Chinese (Ch) and the target language is English (En), the marginal word probability distribution in English is

$$P_{\mathrm{En}}(w) = \sum_k \phi_k^{\mathrm{En}}(w)\theta_k^{\mathrm{Ch}} \tag{5.57}$$

where θ_k is the prior for the k^{th} topic and $\varphi_k(w)$ is the probability assigned to word w by the k^{th} latent topic. As we can see, topic priors are determined by the source language, whereas the topic-dependent word probability distribution is determined by the target language. The target-language marginals are then incorporated into the target-language model as follows:

$$P_{\mathrm{target}}(w|h) \propto \left(\frac{P_{\mathrm{bLSA}}(w)}{P_{\mathrm{base}}(w)}\right)^{\beta} P_{\mathrm{base}}(w|h) \tag{5.58}$$